

Lab 1: Python Refresher for Advanced Machine Learning

Course-	M.Tech.	Type- Core	Core
Course Code-	ABAI5005L	Course Name-	Advanced Machine Learning
Year-	2026-27	Semester-	ODD
Date-	TBD	Batch-	1

Scope of this lab: This lab provides a refresher on core Python programming constructs — variables, control flow, functions, and data structures — before introducing the two libraries fundamental to machine learning workflows, NumPy and Pandas, along with basic visualization using Matplotlib. Students close the lab by preparing a small dataset for a machine learning pipeline through normalization and correlation analysis. Duration: 2 Hours. Dataset: A small synthetic student dataset (Name, Marks, Height, Weight) generated within the notebook. Software: Python 3.x, Jupyter Notebook / Google Colab.

Learning Objectives

After completing this laboratory, students will be able to:

- Understand Python program flow and control structures (conditionals, loops).
- Define and use functions, including functions with multiple arguments and return values.
- Work with core data structures — lists, tuples, dictionaries and sets — and use list comprehensions.
- Perform numerical computing with NumPy: array creation, indexing, broadcasting, statistics and matrix operations.
- Manipulate and explore tabular data using Pandas: filtering, grouping and derived columns.
- Create basic visualizations using Matplotlib to explore data distributions and relationships.
- Prepare a dataset for machine learning through normalization and correlation analysis.

Background

Part A: Python Fundamentals and Control Flow

Python is a dynamically typed, interpreted language, so variables do not require explicit type declarations — the same name can be reassigned to values of different types. Arithmetic operators follow standard precedence rules, and conditional statements (if-elif-else) allow the program to branch based on data. Control flow of this kind underlies far more than toy examples: training loops, early-stopping checks and hyperparameter search routines in ML pipelines are all built from the same conditional and iterative constructs practiced here.

Part B: Loops and Iteration

for and while loops repeat a block of code over a sequence or until a condition is met. In machine learning, this pattern reappears as the epoch loop that repeatedly passes over training data, or the mini-batch loop that iterates over subsets of a dataset during gradient-based optimization.

Part C: Functions

Functions package logic into reusable, testable units, and can take multiple arguments, default values and return values. Many classical ML algorithms are, at their core, a handful of small functions composed together — a distance metric, a normalization step and a decision rule. Writing functions such as `euclidean_distance()` and `normalize()` in this lab previews the building blocks behind algorithms such as k-Nearest Neighbours.

Part D: Data Structures

Lists (ordered, mutable), tuples (ordered, immutable), dictionaries (key-value pairs) and sets (unordered, unique elements) each suit different tasks. Dictionaries are commonly used to store model configurations or hyperparameters; sets are useful for de-duplicating feature names; list comprehensions offer a concise way to transform a collection in a single line.

Part E: Numerical Computing with NumPy

NumPy's ndarray supports vectorized operations that apply an operation to an entire array without an explicit Python loop, which is both faster and closer to how linear algebra is expressed mathematically. Broadcasting rules let arrays of different shapes interact, and operations such as the dot product and matrix transpose underpin techniques used throughout ML, from the normal equation in linear regression to dimensionality reduction methods such as PCA.

Part F: Data Manipulation and Visualization with Pandas and Matplotlib

The Pandas DataFrame is the standard tabular structure that feeds into most ML pipelines, including scikit-learn. Filtering rows, grouping by a category and adding derived columns are everyday steps in preparing a dataset. Matplotlib is used for exploratory data analysis — line, bar, histogram and scatter plots help reveal trends, skewed distributions and outliers before any model is trained.

Part G: From Raw Data to ML-Ready Data

Features measured on different scales (e.g., height in centimetres and weight in kilograms) can bias distance-based algorithms and slow the convergence of gradient-based optimization, so numeric features are typically normalized or standardized before modeling. Computing the correlation between two features is a quick first check of how strongly they are related, which is often the first step in feature selection.

Dataset

For Part G and the mini ML exercise, students will use a small synthetic dataset representing student attributes (Name, Marks, Height, Weight), generated within the notebook using NumPy's random number generation — no external download is required.

Students should identify:

- Numerical vs. categorical attributes in the generated dataset
- A sensible derived column (e.g., Result: Pass/Fail based on Marks)
- Which columns are candidates for normalization before use in an ML model

Required Libraries

Library	Purpose
numpy	Numerical computing: array creation, statistics, random number generation, matrix operations
pandas	Creating and manipulating DataFrames — filtering, grouping, derived columns
matplotlib.pyplot	Line, bar, histogram and scatter plot visualization
math (optional)	Basic mathematical functions to support custom distance/normalization functions

Question 1: Python Fundamentals and Decision Making

Objective: Practice variables, data types, input/output, arithmetic operators and conditional branching.

Instructions:

- Declare variables of different types (int, float, str, bool) and print their types using type().

- Take a student's marks as input and classify the grade using if-elif-else (e.g., A/B/C/F).
- Evaluate a few arithmetic expressions and observe operator precedence.

Hint — Useful functions/classes:

- `input()`, `print()`, `type()`
- `if-elif-else`

Question 2: Loops and Functions

Objective: Practice iteration and modular code, and build small functions that reappear later as building blocks of ML algorithms.

Instructions:

- Print numbers 1 to N using a for loop, then print only the even numbers.
- Compute the factorial of a number using a loop.
- Define functions `square(x)` and `cube(x)`.
- Define `euclidean_distance(point1, point2)` to compute the distance between two n-dimensional points.
- Define `normalize(values)` that scales a list of numbers to the range [0, 1].

Hint — Useful functions/classes:

- `def`, `return`, `range()`
- `math.sqrt()` or manual exponentiation
- `min()`, `max()`

Question 3: Data Structures

Objective: Practice using and transforming Python's core data structures.

Instructions:

- Create a list of student names and marks, and sort it by marks.
- Create a dictionary mapping student name to marks; retrieve and update values.
- Create a set from a list of feature names containing duplicates to demonstrate de-duplication.
- Use a list comprehension to derive a list of letter grades from the marks list.

Hint — Useful functions/classes:

- `list.sort()`, `sorted()`
- dict methods: `keys()`, `values()`, `items()`
- `set()`, list comprehension syntax

Question 4: Numerical Computing with NumPy

Objective: Practice array-based numerical computing and see its performance and mathematical role in ML.

Instructions:

- Create 1D and 2D NumPy arrays; inspect their shape, dtype and size.
- Compute mean, standard deviation, minimum, maximum and sum, including along specific axes.
- Perform matrix operations: transpose and matrix multiplication (dot product).
- Generate random numbers using `numpy.random`, and compare the time taken for a vectorized operation versus an equivalent Python loop.

Hint — Useful functions/classes:

- `np.array()`, `np.mean()`, `np.std()`

- `np.dot()`, `array.T`
- `np.random.rand()`, `%timeit`

Question 5: Data Manipulation and Visualization with Pandas

Objective: Practice building, filtering and visualizing tabular data.

Instructions:

- Create a DataFrame with columns Name, Marks, Height and Weight.
- Filter students who scored above a chosen threshold, and add a derived Result column (Pass/Fail).
- Group the data by Result and compute summary statistics for each group.
- Create a line plot, a bar plot and a histogram to visualize different aspects of the data.

Hint — Useful functions/classes:

- `pd.DataFrame()`, *boolean indexing*
- `groupby()`, `describe()`
- `plt.plot()`, `plt.bar()`, `plt.hist()`

Question 6: Mini ML Exercise — Preparing Data for Machine Learning

Objective: Apply normalization and correlation analysis to prepare data for a downstream ML model.

Instructions:

- Normalize the Height and Weight columns using min-max scaling (reuse your `normalize()` function from Question 2, or use NumPy directly).
- Compute the correlation between Height and Weight (and, optionally, Marks).
- Create a scatter plot of normalized Height vs. Weight and comment on any visible relationship.
- Discuss why normalization is generally necessary before using distance-based or gradient-based ML algorithms.

Hint — Useful functions/classes:

- $(x - \min) / (\max - \min)$
- `DataFrame.corr()`
- `plt.scatter()`

Key Questions for Discussion

- Why is vectorized NumPy computation faster than an equivalent Python loop?
- Why must numeric features often be normalized before they are used by a distance-based or gradient-based model?
- How do dictionaries differ from lists in typical ML use — e.g., storing hyperparameters versus storing samples?
- What does a strong positive or negative correlation between two features suggest about feature selection?
- Why are functions like `euclidean_distance()` and `normalize()` building blocks of ML algorithms rather than just generic utilities?

Expected Deliverables

Students should submit:

- Python notebook containing all completed exercises (Questions 1-6).

- Working implementations of `square()`, `cube()`, `euclidean_distance()` and `normalize()`.
- DataFrame-based analysis with filtering, grouping and the derived Result column.
- All required plots: line, bar, histogram and scatter.
- Correlation analysis with a short written interpretation of the Height-Weight relationship.
- Completed Expected Analysis / self-assessment table.